Introduction to Applied Machine Learning

13: Generative Models and Diffusion

🎓 Machine Learning Adventure 🚀

1. 🤖 Introduction to Machine Learning
2. 🧠 Artificial Neural Networks and Computer Vision
3. 🔤 Transformers and Language Models
4. 🎨 GANs and Diffusion for Image Generation
5. 🎮 Reinforcement Learning and Behavior
6. 🏆 Final Projects

Last Time: Multimodal Models with Image Inputs

How do modern large “language” models also handle image inputs?

What is this a picture of?

This is a picture of a black and white cat lounging on a piece of furniture, likely a chair or couch. The cat has a tuxedo-like pattern with a white chest, legs, and face markings.

ChatGPT accessed at chatgpt.com on 10/5/2025

Crucial Elements:

1. Vision Transformers (ViT), and
2. Contrastive Learning

Generative Image Models

DALL·E 2 · An expressive oil painting of a basketball player dunking, depicted as an explosion of a nebula.

DALL·E 3 · An expressive oil painting of a basketball player dunking, depicted as an explosion of a nebula.

Example: DallE 3, integrated into ChatGPT https://openai.com/index/dall-e-3/

Other Applications: Super-resolution

From https://today.duke.edu/2020/06/artificial-intelligence-makes-blurry-faces-look-more-60-times-sharper. Work by Duke researchers Sachit Menon, Alex Damian, McCourt Hu, Nikhil Ravi and Dr. Cynthia Rudin.

Other Applications: Image Completion

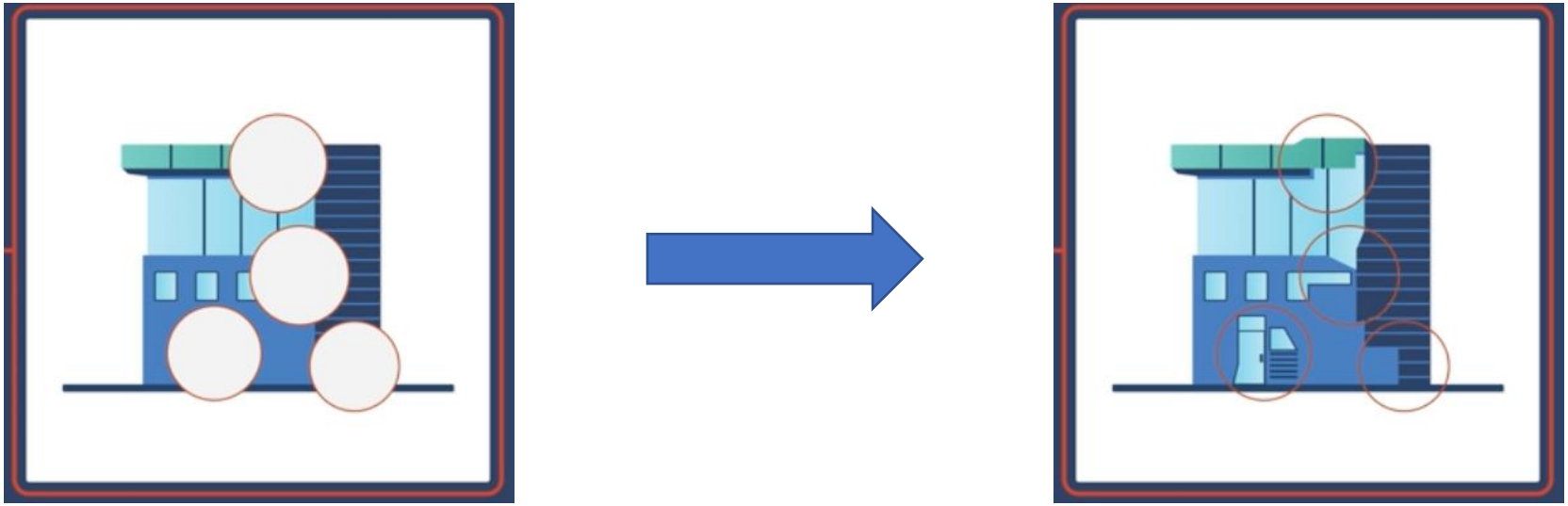
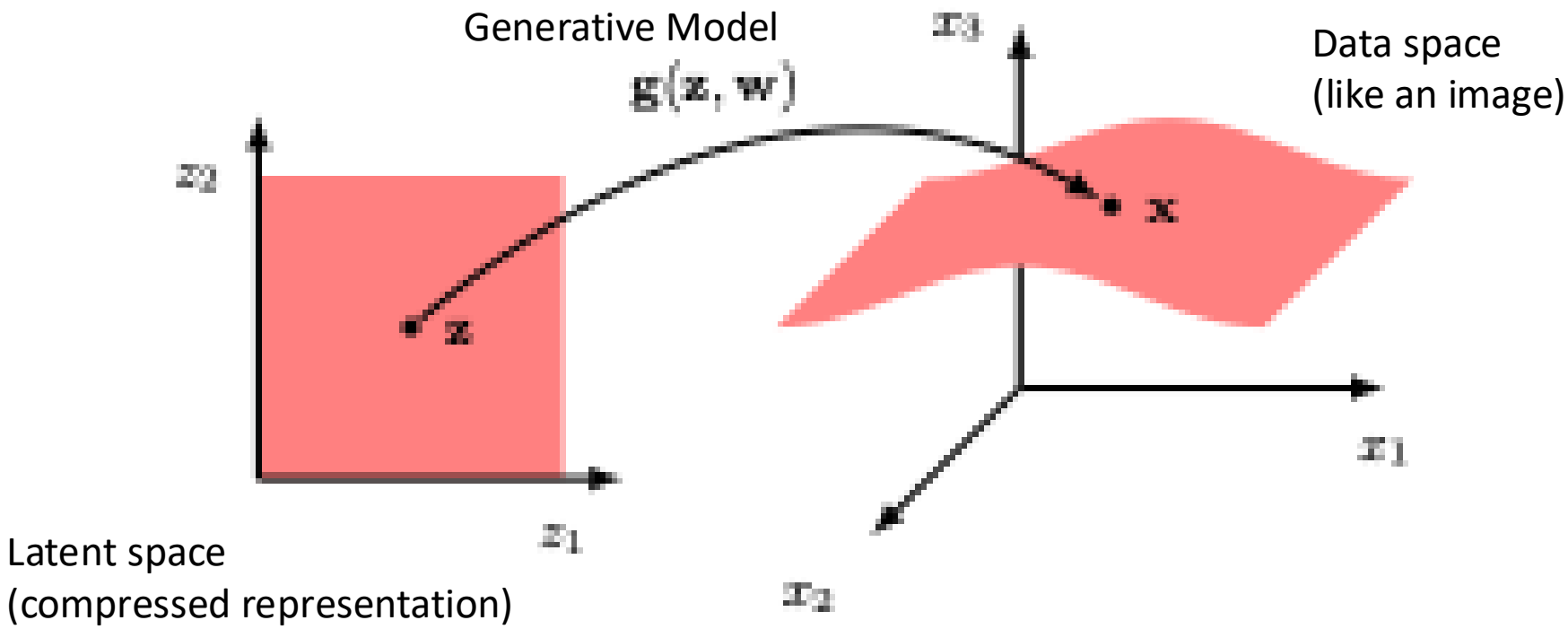


Illustration from CoModGAN, published in ICLR 2021. GitHub page: https://github.com/zsyzzsoft/co-mod-gan.

Latent Variable Modeling

Assume there is a **low-dimensional latent** representation of the data ($\boldsymbol{z}$) and learn the weights ($\boldsymbol{w}$) of a **generative** model $\boldsymbol{g}(\boldsymbol{z}, \boldsymbol{w})$, a function which maps to the data space ($\boldsymbol{x}$).

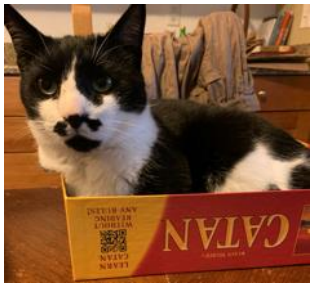


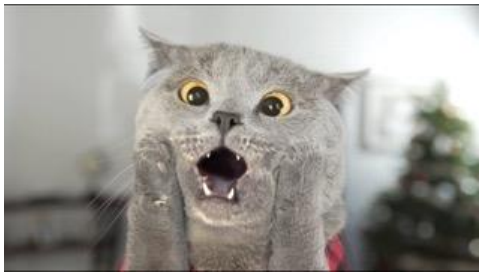
Accuracy?

What counts as a picture “of a cat”?

No notion of the ground truth (or correct answer).

Seems pretty good



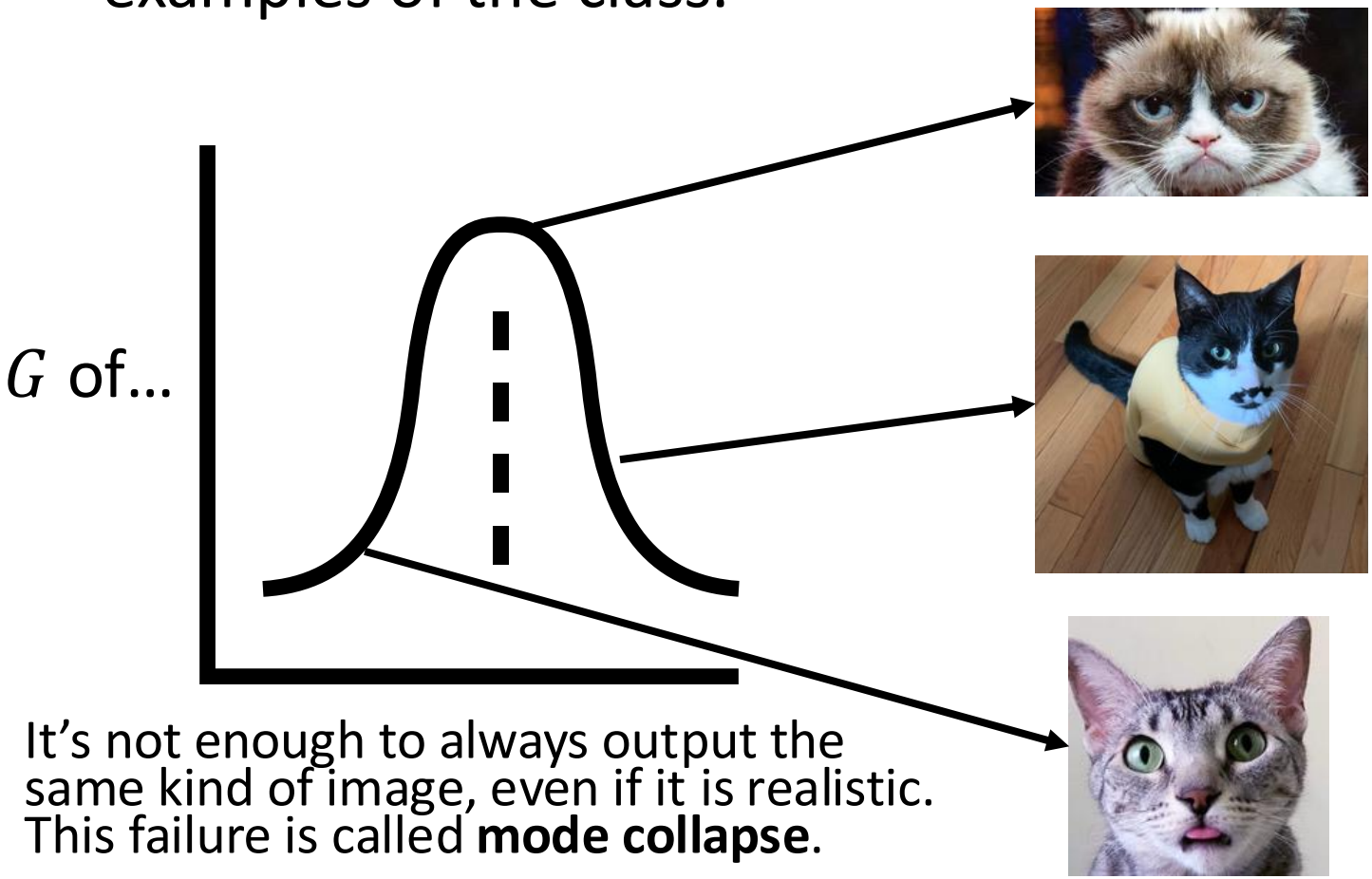


Not so good

But images should be “realistic” as compared to training data.

Diversity

Different random input should generate diverse examples of the class.

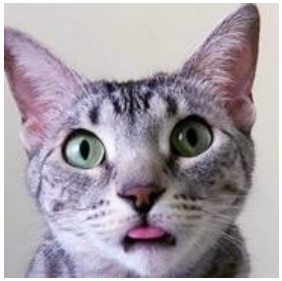


It's not enough to always output the same kind of image, even if it is realistic. This failure is called **mode collapse**.

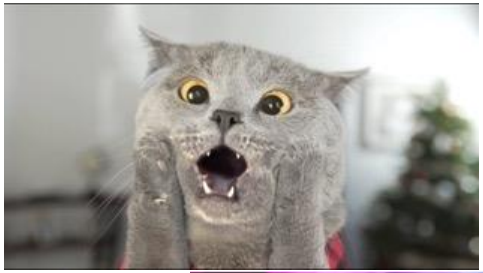
Generalization

G is trained on real data but should generate new unseen examples. Should not just memorize training examples.

G trained on…

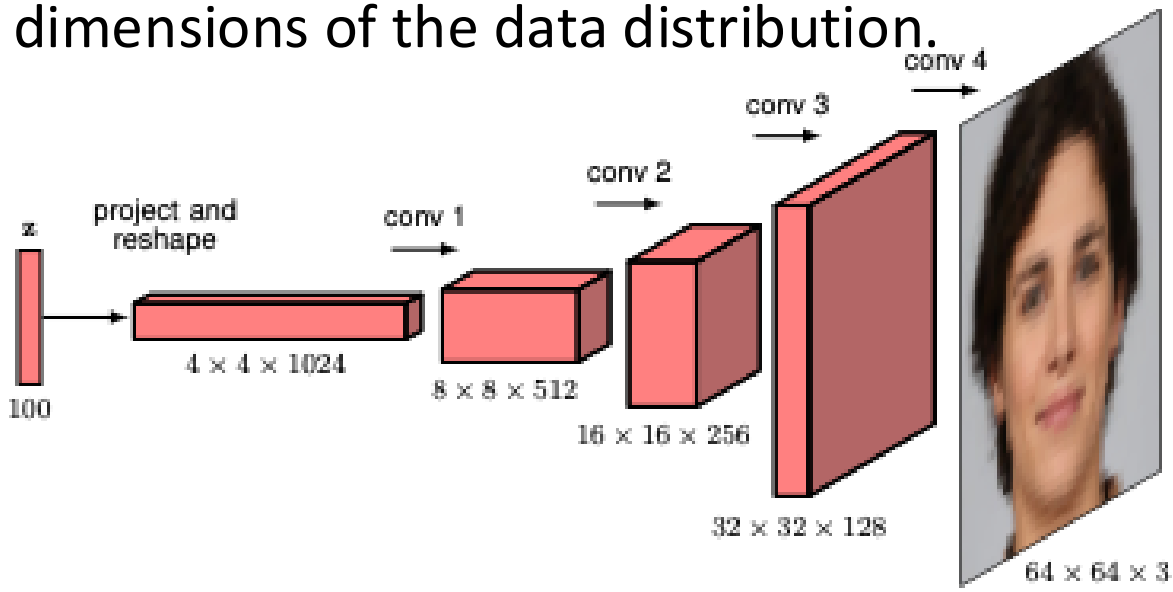


Can generate…



Generator as a Neural Network

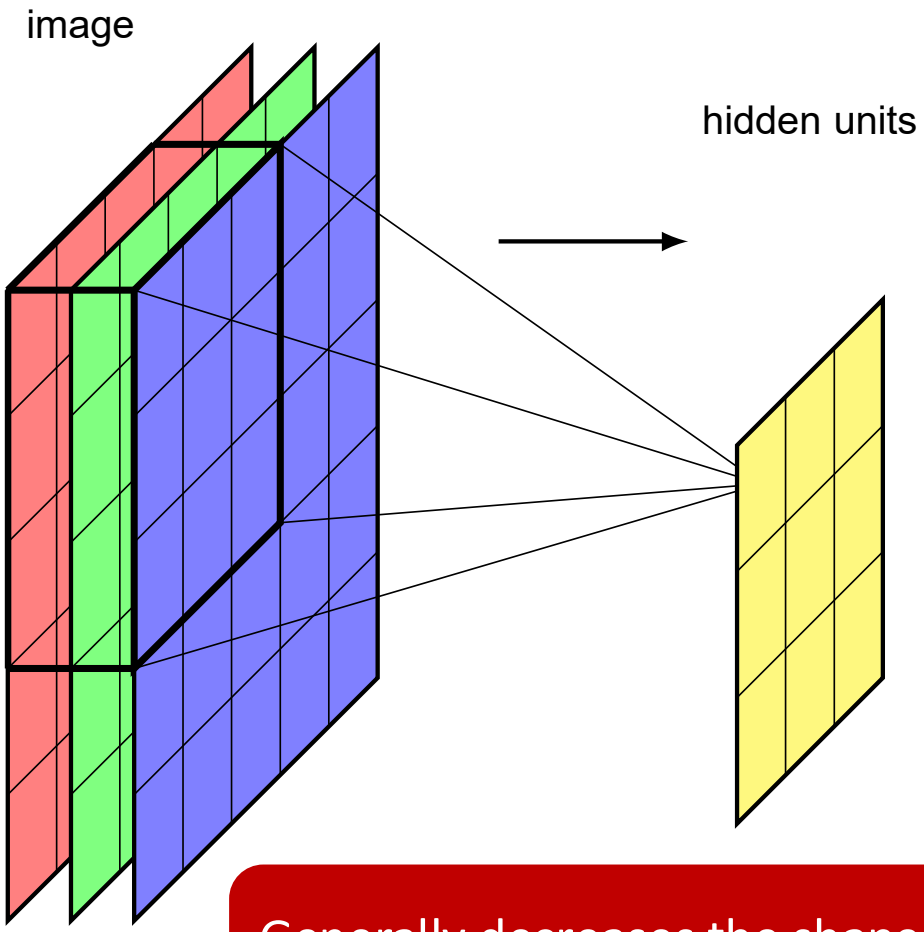
The generator $\boldsymbol{g}(\boldsymbol{z}, \boldsymbol{w})$ maps vectors from a latent space to the dimensions of the data distribution.

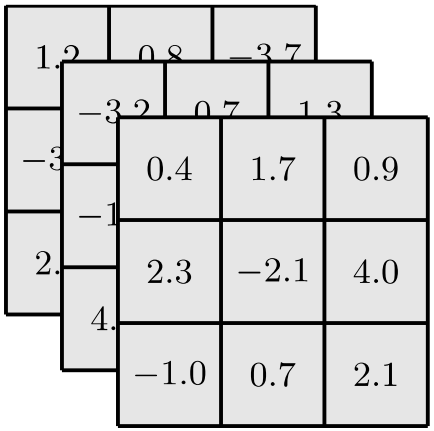


Two approaches to training diverse and realistic generators:

1. GANs (adversarial training)
2. Diffusion (learning to denoise)

Standard 2D Convolutions





For n input channels $I_1, I_2, \ldots, I_n$, kernels $K_1, K_2, \ldots, K_n$, and bias b…

$$C_{\text{out}} = b + \sum_{j=1}^{n} I_j * K_j$$

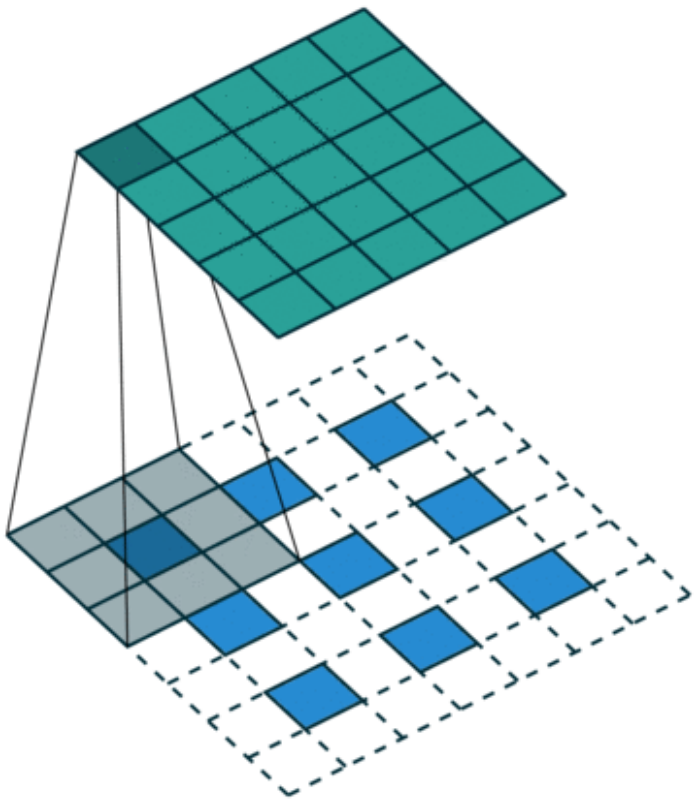
Generally decreases the shape of the input!

“Doconvolutional” Layers: Transposed Convolutions

How can a convolution increase spatial resolution / be used as for a learnable up-sampling procedure?

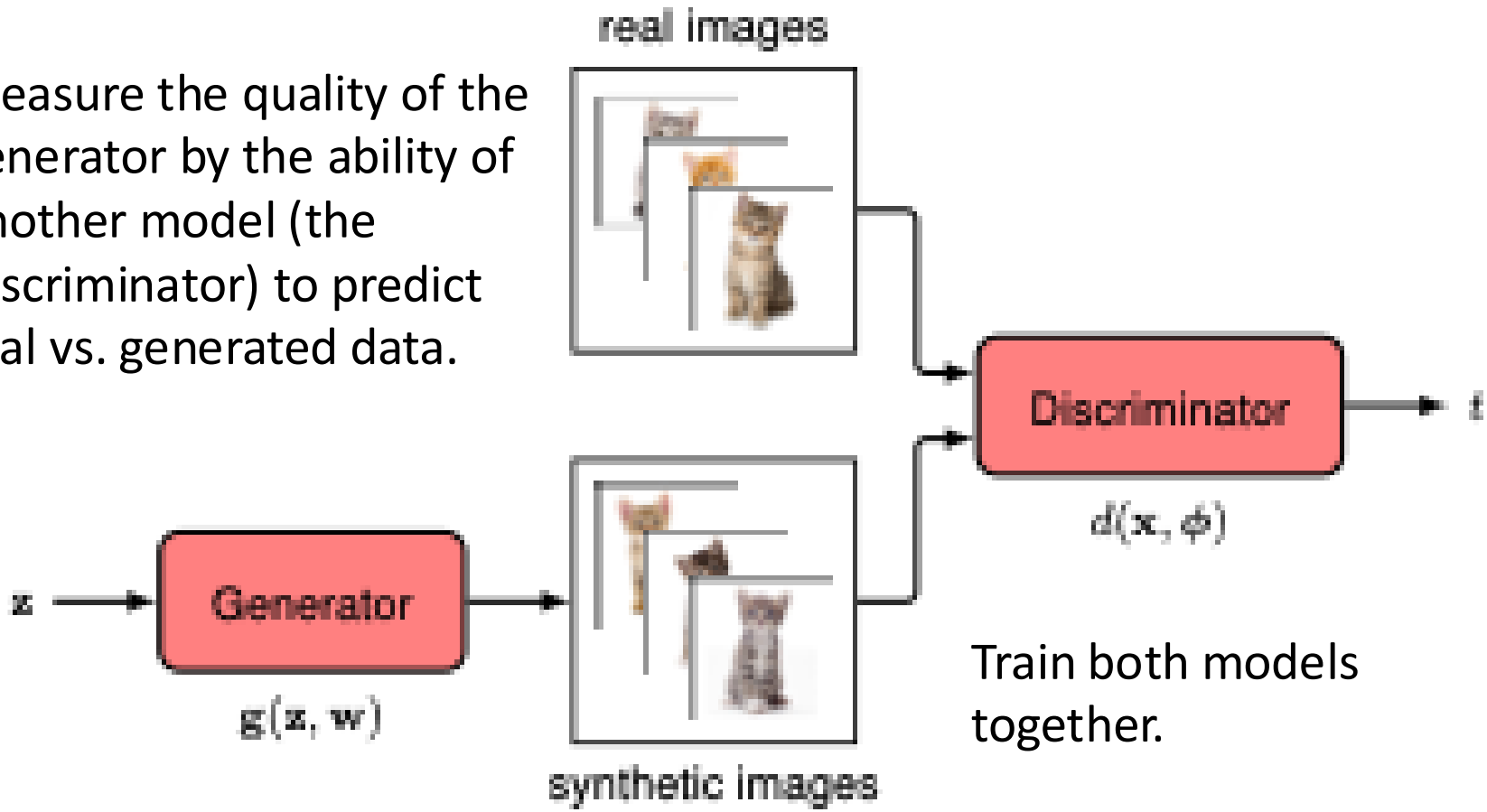
The **transposed convolution** with stride S and $k \times k$ kernel:

1. “Dilates” the input by adding S-1 0s between every pair of adjacent input values.
2. May pads the dilated input by adding p 0s on all sides.



Generative Adversarial Network (GAN): Adversarial Training

Measure the quality of the generator by the ability of another model (the Discriminator) to predict real vs. generated data.

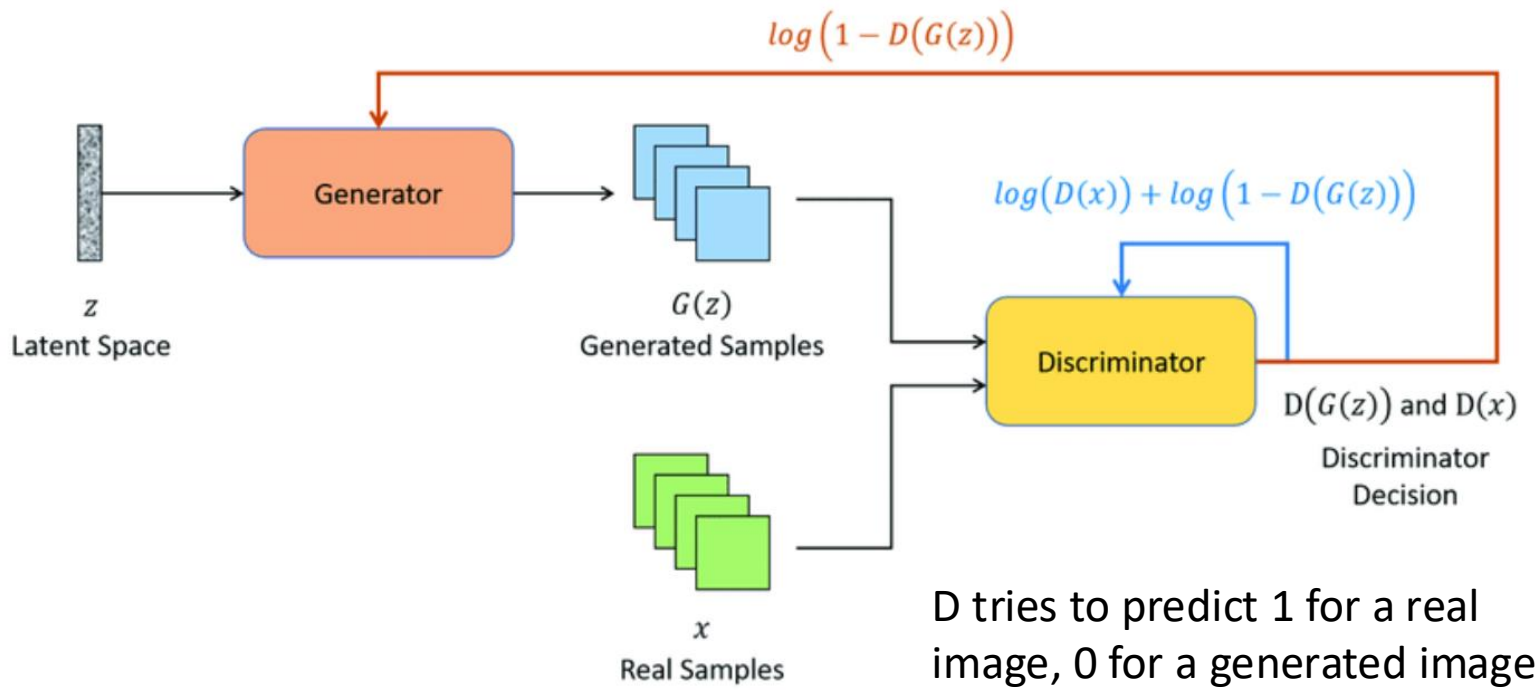


Train both models together.

Learning a Generator and Discriminator Together

The loss function becomes ***adversarial*** – G and D have different objectives in a 2-player game.

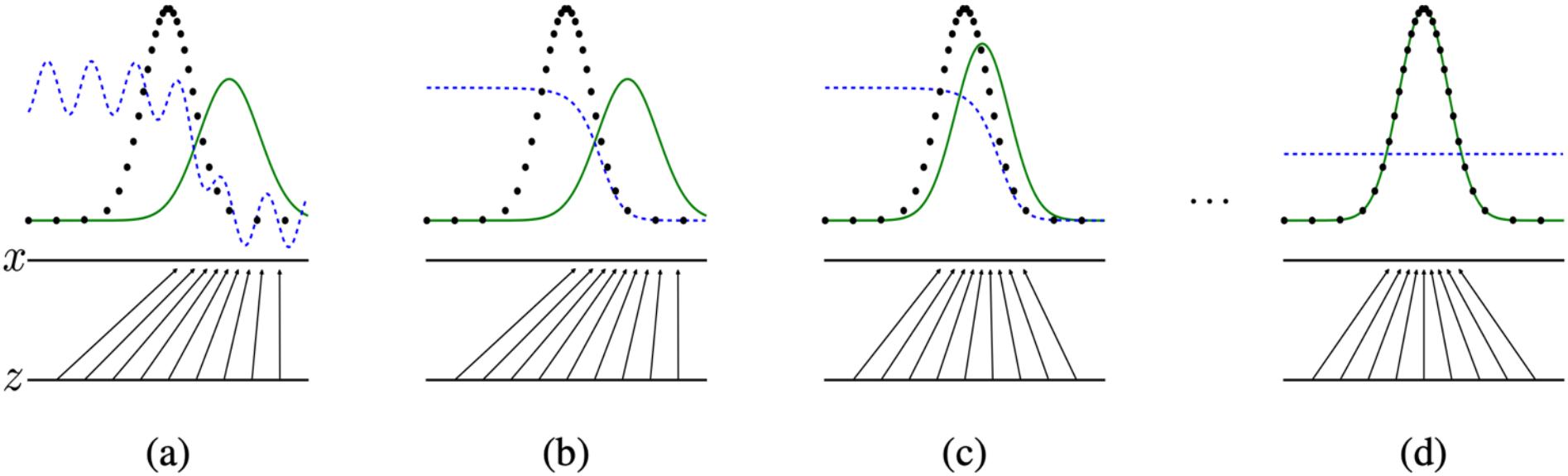
$$\min_{\theta_G} \max_{\theta_D} \mathbb{E}_{x\sim\text{Data}}[\log(D(x,\theta_D))] + \mathbb{E}_{z\sim p_z(z)}\left[\log\left(1 - D(G(z,\theta_G),\theta_D)\right)\right]$$



D tries to predict 1 for a real image, 0 for a generated image.

Conceptual Example

Data: Black, dotted
D: Blue, dashed
G: Green, solid



Before

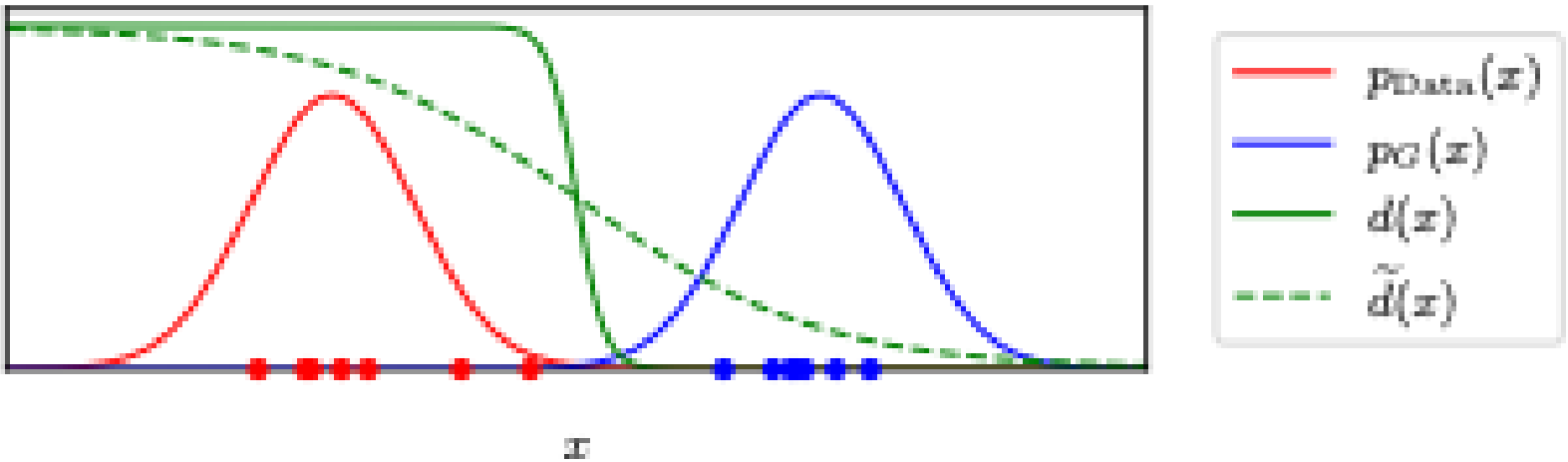
Gradient updates to Discriminator

Gradient updates to Generator

At convergence

GAN Training Challenges: Initialization and Convergence

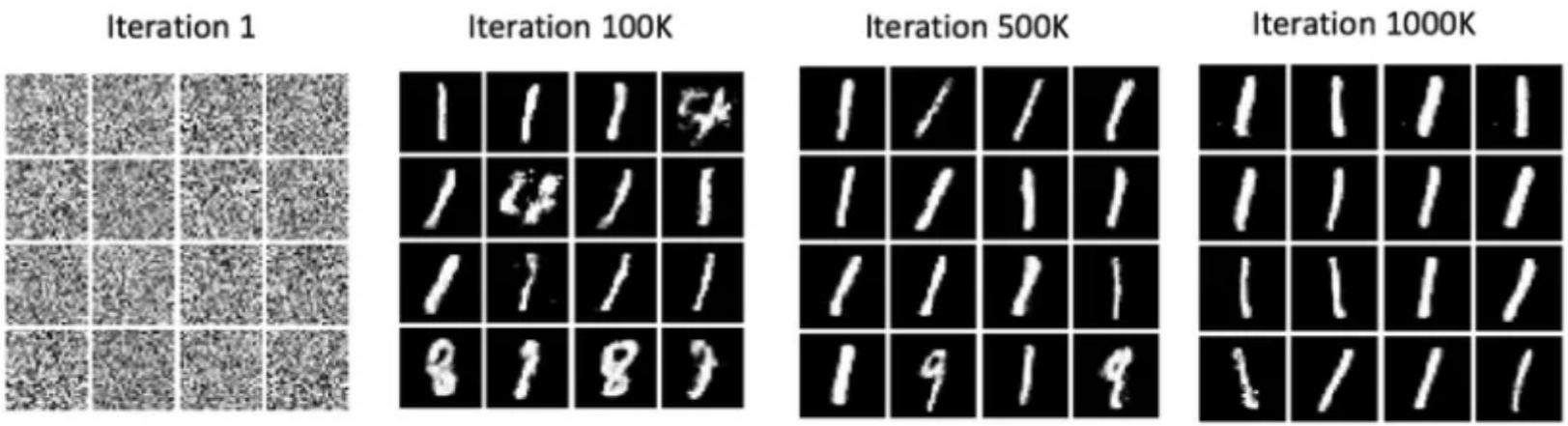
The initial generative distribution can be very far from the data distribution, leading to ≈ 0 gradient for training the discriminator.



No guarantee of convergence towards an equilibrium of the game, and convergence often fails in practice.

GAN Training Challenges: Mode Collapse

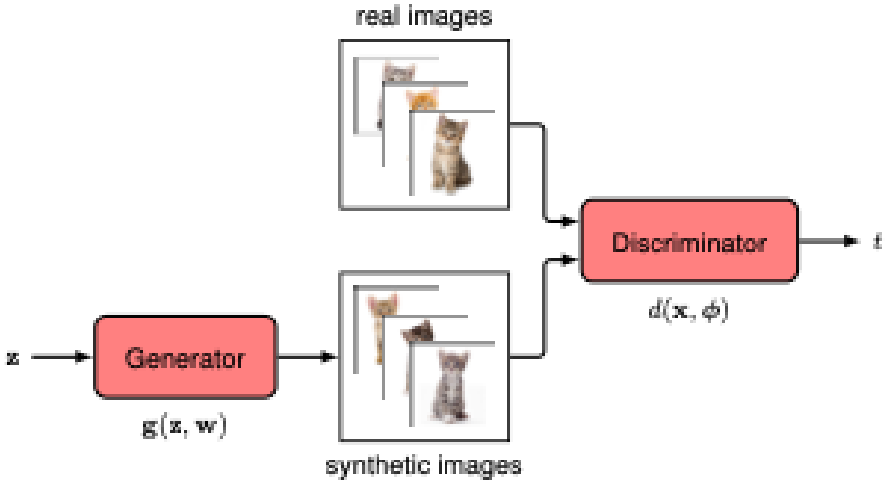
Mode Collapse: A bad local optimum in which the generator only produces a small number, or even a single type of example.



This makes the task difficult for the discriminator (achieves realism) but fails on diversity.

From GANs to Diffusion

GANs enable training an efficient and realistic generator using a relatively small amount of data.



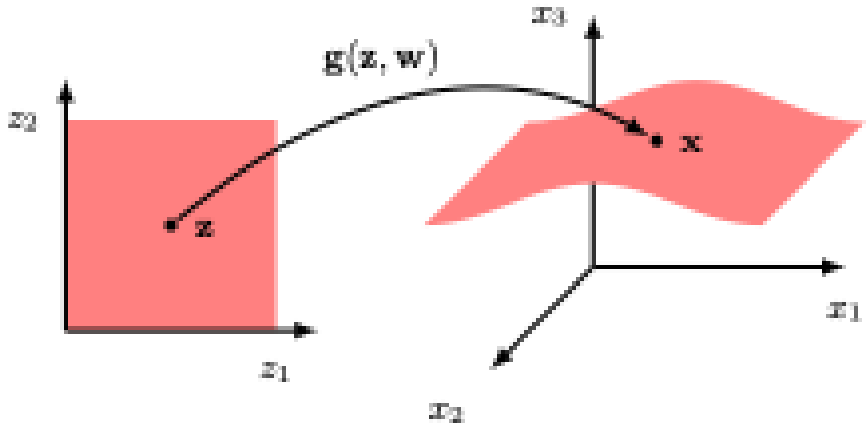
But GANs suffer from:

- Unstable training, sensitive to hyperparameters and not guaranteed to converge
- Mode collapse and limited diveristy

GANs are still relevant for efficient inference applications, but more modern systems that prioritize stable training and diverse generations are usually built on Diffusion.

Recall: Latent Variable Modeling and a Process Perspective

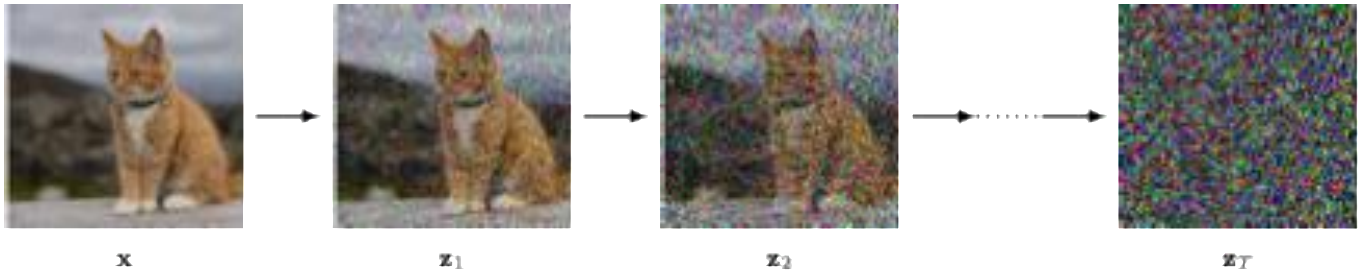
A generator $\boldsymbol{g}(\boldsymbol{z}, \boldsymbol{w})$ that maps from latent to data space in a single step is a difficult predictive task.



Instead, what if we viewed this as a process of iteratively projecting into a progressively higher quality (less random) space?

Diffusion Process

Imagine iteratively adding Gaussian random noise to an image: A **forward diffusion process** $q(\boldsymbol{z}_t \mid \boldsymbol{z}_{t-1})$.

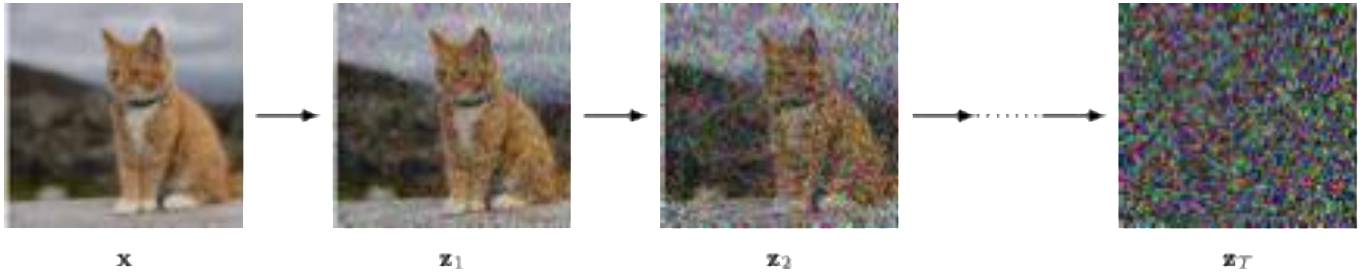


The **reverse diffusion process** $q(\boldsymbol{z}_{t-1} \mid \boldsymbol{z}_t)$ would iteratively remove the noise

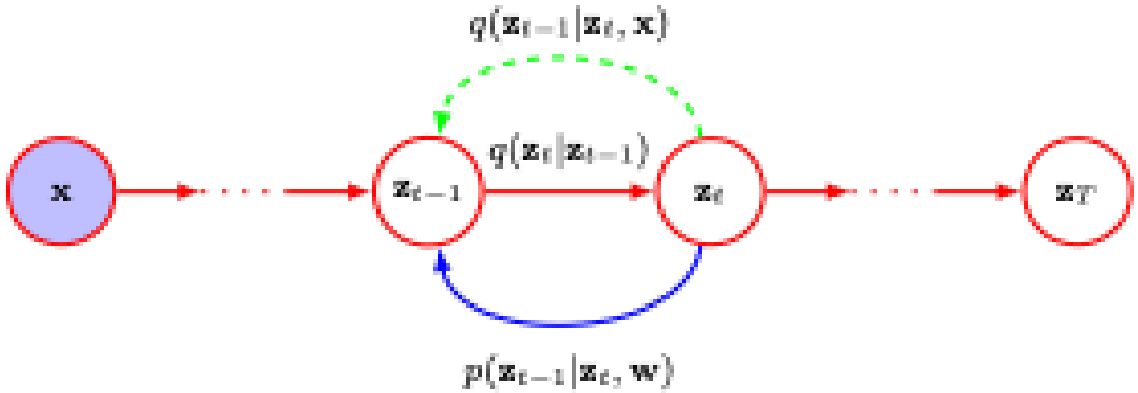
The forward process just requires adding noise, but we can train a model of the reverse process to use as a generator.
This is the approach taken by popular image generators such as Stable Diffusion, DALL-E, etc.

Diffusion Model

Given a **forward diffusion process** $q(\boldsymbol{z}_t \mid \boldsymbol{z}_{t-1})$.



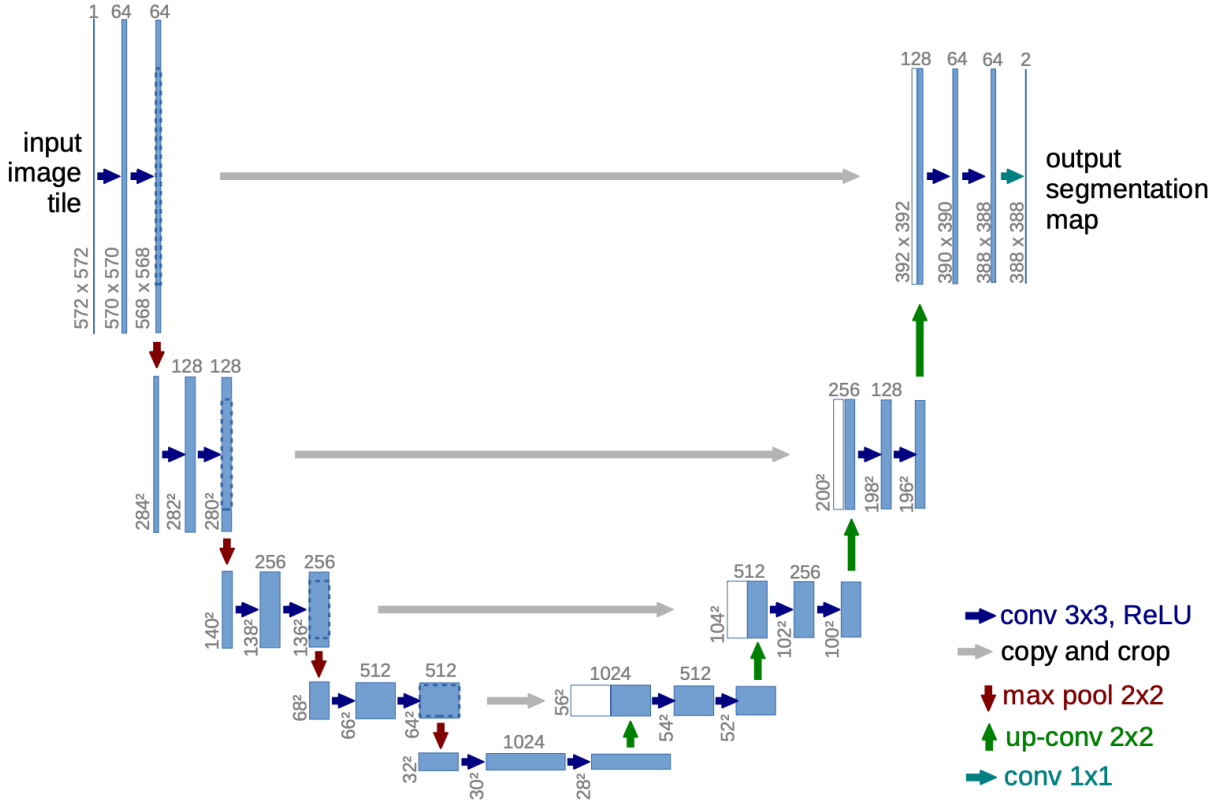
Want to learn a model $p(\boldsymbol{z}_{t-1} \mid \boldsymbol{z}_t, \boldsymbol{w})$ with parameters $\boldsymbol{w}$ of the reverse diffusion process to use as a generator.



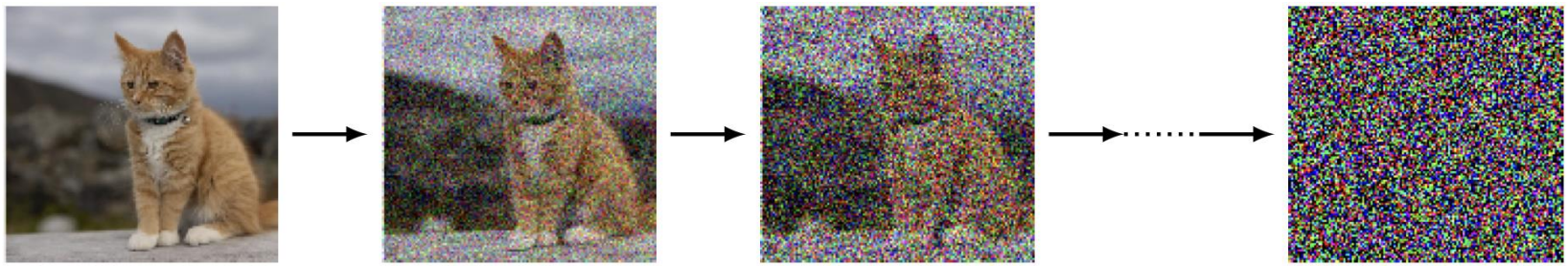
Unlike in GANs, each prediction $p(\boldsymbol{z}_{t-1} \mid \boldsymbol{z}_t, \boldsymbol{w})$ has a correct answer, no adversarial training needed.

Model Architecture: UNet

The **UNet** architecture has inputs and outputs of the same shape, with convolutions and pooling for downsampling (encoding) and upsampling (decoding), with **skip connections** (copy and concatenate to input).



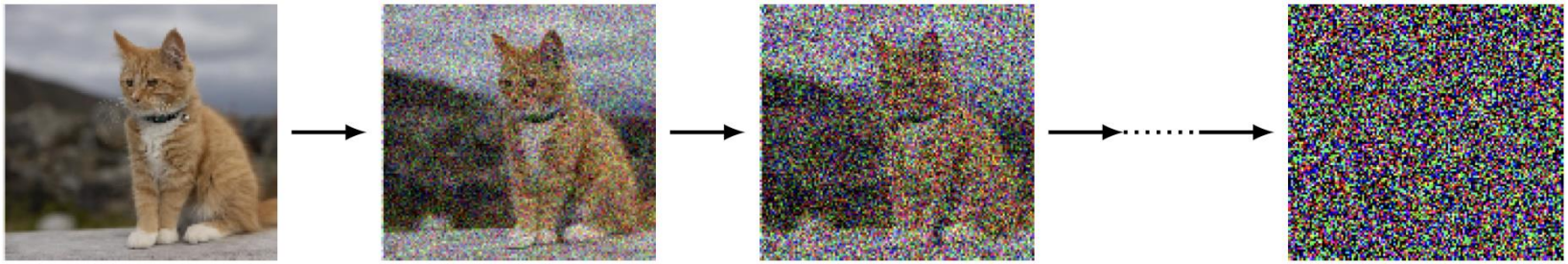
Forward Process Details



$\boldsymbol{x}_t = \sqrt{1-\beta_t}\boldsymbol{x}_{t-1} + \sqrt{\beta_t}\boldsymbol{\epsilon}$ where…

- $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
  - That is, use unit normal noise independently in every element
- $\{\beta_1, \beta_2, \dots, \beta_T\}$ is the **noise schedule;** T is the number of steps.
  - Common to use $\beta_1, < \beta_2 < \cdots < \beta_T$
  - larger the β the greater the noise
- The exact formula is chosen to preserve variance across time steps and so that as $T \to \infty$, $\boldsymbol{x}_T$ becomes pure Gaussian noise.

Reparametrizing the Forward Process



- In minibatch SGD training, we would like to train on an *independent random* batch of datapoints.
- To sample a random point along the forward process, we can reparametrize:

$$\boldsymbol{x}_t = \sqrt{1-\beta_t}\boldsymbol{x}_{t-1} + \sqrt{\beta_t}\boldsymbol{\epsilon} = \sqrt{\alpha_t}\boldsymbol{x} + \sqrt{1-\alpha_t}\boldsymbol{\epsilon}$$

- Where $\alpha_t = \prod_{\tau=1}^{t}(1-\beta_\tau)$

Reparametrizing the Generator (Reverse Process Model)

- Instead of a model to predict the next image in the reverse process, more common to predict the *noise that was added:* $\boldsymbol{g}(\boldsymbol{z}_t, \boldsymbol{w}, t)$.
- Then (intuitively) we can just remove the next image in the reverse process.
  - A bit more complicated because of the scaling in the forward process.

$$\boldsymbol{x}_t = \sqrt{1-\beta_t}\boldsymbol{x}_{t-1} + \sqrt{\beta_t}\boldsymbol{\epsilon} = \sqrt{\alpha_t}\boldsymbol{x} + \sqrt{1-\alpha_t}\boldsymbol{\epsilon}$$

Diffusion Inference/Generation

Algorithm 20.2: Sampling from a denoising diffusion probabilistic model

Input: Trained denoising network $\mathbf{g}(\mathbf{z}, \mathbf{w}, t)$
Noise schedule $\{\beta_1, \ldots, \beta_T\}$
Output: Sample vector $\mathbf{x}$ in data space

$\mathbf{z}_T \sim \mathcal{N}(\mathbf{z}|\mathbf{0}, \mathbf{I})$ `// Sample from final latent space`
for $t \in T, \ldots, 2$ **do**
  $\alpha_t \leftarrow \prod_{\tau=1}^{t}(1-\beta_\tau)$ `// Calculate alpha`
  `// Evaluate network output`
  $\boldsymbol{\mu}(\mathbf{z}_t, \mathbf{w}, t) \leftarrow \frac{1}{\sqrt{1-\beta_t}}\left\{\mathbf{z}_t - \frac{\beta_t}{\sqrt{1-\alpha_t}}\mathbf{g}(\mathbf{z}_t, \mathbf{w}, t)\right\}$
  $\boldsymbol{\epsilon} \sim \mathcal{N}(\boldsymbol{\epsilon}|\mathbf{0}, \mathbf{I})$ `// Sample a noise vector`
  $\mathbf{z}_{t-1} \leftarrow \boldsymbol{\mu}(\mathbf{z}_t, \mathbf{w}, t) + \sqrt{\beta_t}\boldsymbol{\epsilon}$ `// Add scaled noise`
end for
$\mathbf{x} = \frac{1}{\sqrt{1-\beta_1}}\left\{\mathbf{z}_1 - \frac{\beta_1}{\sqrt{1-\alpha_1}}\mathbf{g}(\mathbf{z}_1, \mathbf{w}, t)\right\}$ `// Final denoising step`
return $\mathbf{x}$

Reasoning about T: Generation requires T forward passes through the generator model g. Larger values of T tend to reduce the variance of each step of the diffusion process and improve quality, but smaller values of T decrease the computational complexity of generation.

Diffusion Training

Algorithm 20.1: Training a denoising diffusion probabilistic model

Input: Training data $\mathcal{D} = \{\mathbf{x}_n\}$
Noise schedule $\{\beta_1, \ldots, \beta_T\}$
Output: Network parameters $\mathbf{w}$

---

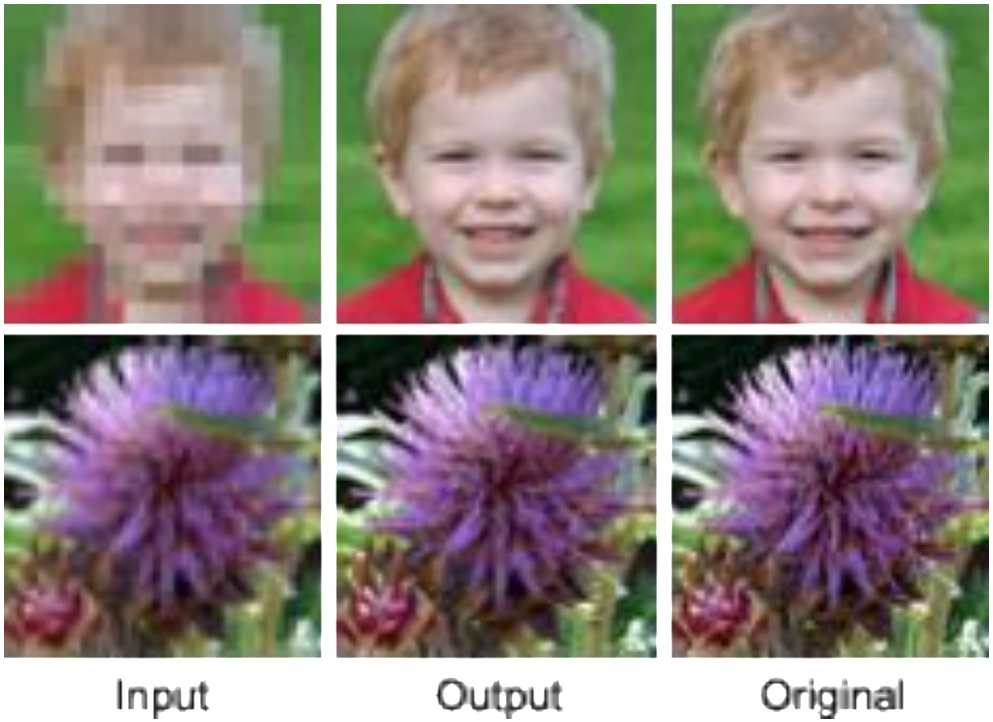
for $t \in \{1, \ldots, T\}$ **do**
  $\alpha_t \leftarrow \prod_{\tau=1}^{t}(1 - \beta_\tau)$ `// Calculate alphas from betas`
end for
repeat
  $\mathbf{x} \sim \mathcal{D}$ `// Sample a data point`
  $t \sim \{1, \ldots, T\}$ `// Sample a point along the Markov chain`
  $\boldsymbol{\epsilon} \sim \mathcal{N}(\boldsymbol{\epsilon}|\mathbf{0}, \mathbf{I})$ `// Sample a noise vector`
  $\mathbf{z}_t \leftarrow \sqrt{\alpha_t}\mathbf{x} + \sqrt{1 - \alpha_t}\boldsymbol{\epsilon}$ `// Evaluate noisy latent variable`
  $\mathcal{L}(\mathbf{w}) \leftarrow \|\mathbf{g}(\mathbf{z}_t, \mathbf{w}, t) - \boldsymbol{\epsilon}\|^2$ `// Compute loss term`
  Take optimizer step
until converged
return $\mathbf{w}$

(pseudocode code shows pure SGD, can minibatch as normal)

Comparing to GAN: No adversarial training, clear predictive target per step, reduced risk of mode collapse leads.

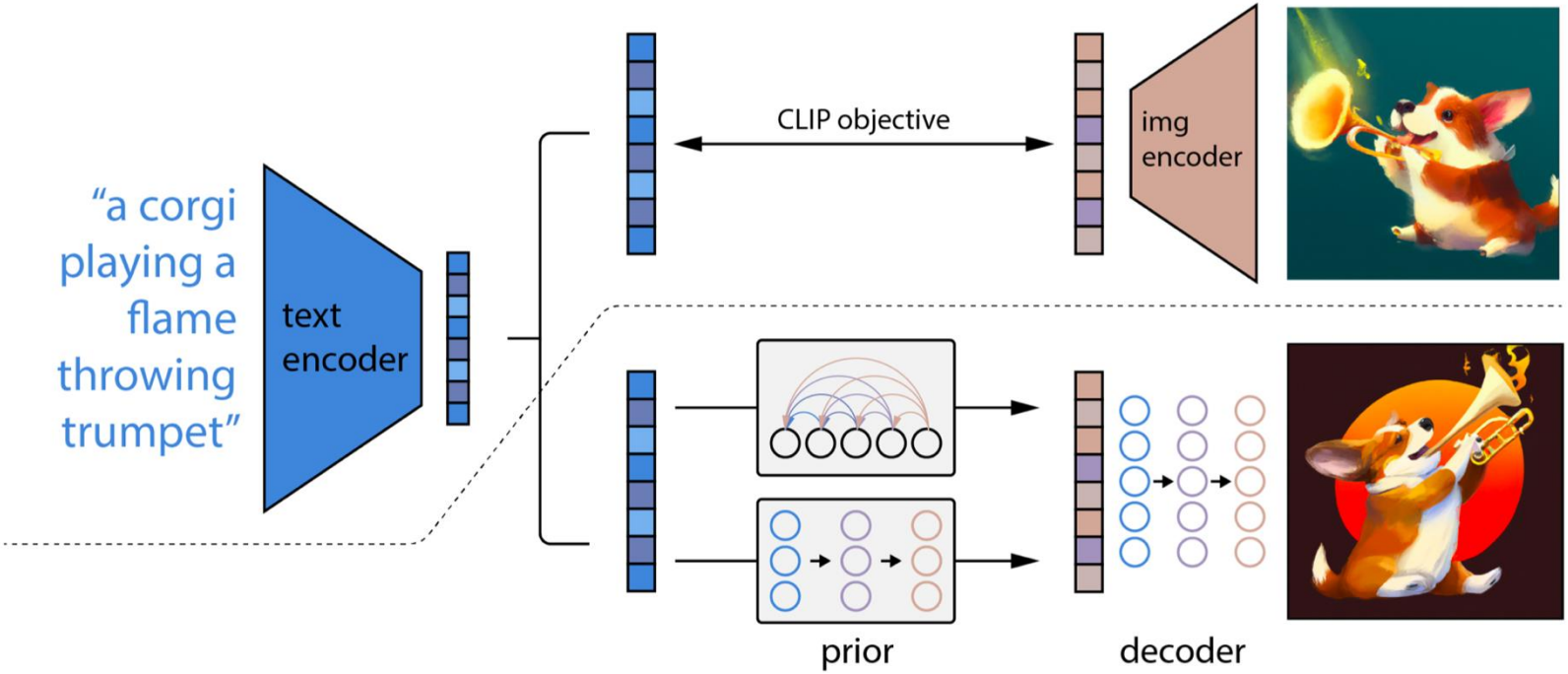
Conditional Generation

We may sometimes want to generate an output **conditional** on some additional context. For example, we might want to generate a de-noised high-resolution image from a low-resolution input.



But what if the input context on which we want to condition is not itself an image?

Text to Image Generation



Coming up next!